

ARC-AGI-3 Execution Dossier:

Autonomous Exploration and Program Synthesis Architectures

The transition from the Abstraction and Reasoning Corpus (ARC-AGI-1 and 2) to ARC-AGI-3 represents a fundamental paradigm shift in the evaluation of artificial general intelligence. While earlier iterations tested static, few-shot pattern induction, ARC-AGI-3 introduces an interactive, hidden-state environment where agents are deployed without explicit instructions, rules, or objective functions¹. The system must autonomously navigate "unknown unknowns" by executing exploratory actions, observing environmental state transitions, hypothesizing underlying mechanics, and executing a path to a hidden goal state¹.

Success in this regime is quantified via Relative Human Action Efficiency (RHAE), a punishing metric calculated as the square of the ratio of human steps to AI steps². An agent that requires 100 steps to solve a puzzle a human solved in 10 steps will receive a score of just 1%².

Consequently, brute-force random exploration is mathematically unviable. The architectural mandate for dominating this challenge within the strict limits of an offline NVIDIA RTX 6000 (48GB VRAM) and a 9-hour runtime utilizing a local Qwen 3.6 27B (FP8 precision) model demands a highly sophisticated, decoupled neurosymbolic execution engine. The language model must not interact directly via token-prediction, but rather act as a high-level cognitive controller that delegates structural manipulation to a specialized Domain Specific Language (DSL), manages uncertainty through Curiosity-Driven Exploration (CDE), and orchestrates test-time compute via Adaptive Branching Monte Carlo Tree Search (AB-MCTS).

Pillar 1: Domain Specific Languages (DSLs) & Program Synthesis

Relying on raw autoregressive pixel-generation from a 27-billion parameter language model for precise, high-dimensional grid operations yields unacceptable hallucination rates.

State-of-the-art ARC systems frame the visual reasoning challenge as an exercise in Program Synthesis, wherein the language model generates symbolic code utilizing a predefined Domain Specific Language (DSL).

Analysis of Optimal Grid-Transformation Primitives

An analysis of open-source solutions from previous ARC winners, specifically the 2020 Kaggle first-place entry by "Icubur," reveals that the most effective approach to DSL construction involves the instantiation of a Directed Acyclic Graph (DAG)⁴. In the Icubur architecture, each node represents a distinct grid state, and edges represent deterministic unary transformations drawn from a library of 142 handcrafted functions⁴. By standardizing colors across input pairs and applying geometric operations iteratively, the solver reduces the infinite pixel space into a constrained topological space⁴.

Modern neurosymbolic frameworks, such as DreamCoder-ARC and the Perceptual Abstraction & Reasoning Language (PeARL), condense these hundreds of micro-operations into robust, higher-order typed primitives that support programmatic abstraction and library learning⁵. For ARC-AGI-3, where grids dynamically react to sequential inputs, the DSL must support both structural manipulation and traversal mechanisms.

Primitive Category	Core Operations	Semantic Role in ARC-AGI-3 Execution
Rigid Transformations	rotate90, rot180, flipX, swapXY	Manipulating contiguous object geometries and structural orientations without altering internal topology ⁶ .
Morphology & Topology	floodFill, fillHoles, makeBorder	Delineating boundaries, identifying enclosed spaces, and clustering identically colored pixels into discrete, movable entities ⁶ .
Cropping & Embedding	ic_embed, cropBackground, leftHalf	Reducing grid dimensionality to focus strictly on localized agent interactions and discarding static environmental noise ⁵ .
Color Mapping	filterCol, recolorMax, invert	Remapping pixel values based on frequency or spatial location to standardize input states across heterogeneous interactive boards ⁴ .
Split & Reduction	split4, split8, pickMaxSize	Decomposing a composite grid into sub-grids based on structural markers, isolating specific interactive components ⁶ .

Traversal Mechanics	rowByRow, snakeTraversal	Systematically scanning state representations to identify hidden interactive triggers or conditional logic ⁷ .
---------------------	--------------------------	---

Efficient Search Space Algorithms

Executing a search over this DSL space is computationally explosive. A naive Breadth-First Search (BFS) will rapidly exhaust system memory and exceed the 9-hour hardware limit. The agent must employ advanced program synthesis search algorithms to maintain tractability.

Top-Down Lambda Calculus (TDLC) with Neural Guidance

TDLC frames program synthesis as a sequence of hole-filling operations, where a program is constructed as an Abstract Syntax Tree (AST). The search initializes with a start symbol and recursively expands non-terminals. The Qwen 3.6 27B model acts as a probabilistic prior, emitting likelihoods for specific DSL productions based on the current grid observation. By enforcing strict type-checking at each AST expansion step (e.g., ensuring a color argument is not erroneously passed to a function expecting a grid object), the search space is aggressively pruned before invalid programs are executed.

Bottom-Up Search with Observational Equivalence Conversely, bottom-up synthesis enumerates sub-programs from basic primitives and progressively builds larger expressions. To optimize this under strict compute budgets, the system hashes the output grids generated by each intermediate sub-program. If two distinct programs produce identical output states on the current ARC-AGI-3 board, they are deemed observationally equivalent. The computationally heavier or deeper program is immediately pruned from the DAG⁴.

Minimum Description Length (MDL) Principle Integration Derived from algorithmic information theory, MDL-based approaches—such as the MADIL framework—evaluate the fitness of a program based on its capacity to compress the state transition⁸. The MDL principle asserts that the optimal model is the one that minimizes the description length of the model itself plus the description length of the data encoded by the model⁸. For an ARC-AGI-3 agent, applying an MDL objective ensures the language model avoids overfitting to spurious pixel variations. Programs that require extensive exception-handling or one-off pixel modifications are heavily penalized, isolating the true, generalized causal rules of the interactive environment⁸.

Pillar 2: Test-Time Compute (TTC) & Search Algorithms

To achieve the necessary RHAЕ scores, the agent cannot execute actions directly against the environment without prior internal validation. The system must utilize Test-Time Compute (TTC) to simulate multiple trajectories, generating and evaluating hypotheses internally before committing to a physical move that increments the action counter².

Adaptive Branching Monte Carlo Tree Search (AB-MCTS)

Standard Monte Carlo Tree Search utilizes a fixed branching factor, which artificially restricts a

language model from leveraging its vast generation diversity. The Adaptive Branching Monte Carlo Tree Search (AB-MCTS) algorithm dynamically balances "going wider" (sampling completely novel hypotheses from the base prompt) versus "going deeper" (refining an existing hypothesis based on error feedback)¹⁰.

The AB-MCTS framework models the generation tree such that each node represents a candidate program or hypothesis. At each decision point, the system employs Thompson Sampling to update a Bayesian posterior evaluating whether to expand a new path or exploit the current one¹³. Two primary parameter update rules exist: the Gaussian (AB-MCTS-M) which utilizes a mixed-effects Bayesian model, and the Beta aggregation (AB-MCTS-A)¹³.

For environments where heuristic scores r are strictly bounded within $[0, 1]$ (such as checking a hypothesis against local ARC grids for pixel-perfect accuracy), the AB-MCTS-A Beta prior is computationally superior and highly sample-efficient¹⁴. The parameter update rules for AB-MCTS-A (Beta) operate as follows:

$$P(r \mid \{r_n\}_{n=1}^N) = B(r \mid \alpha, \beta)$$

$$\alpha = \check{\alpha} + \sum_{n=1}^N r_n$$

$$\beta = \check{\beta} + \sum_{n=1}^N (1 - r_n)$$

Where $\check{\alpha}$ and $\check{\beta}$ are the prior parameters, and N represents the number of sampled evaluations¹⁴. The Qwen LLM acts as the heuristic evaluator, analyzing the predicted grid state resulting from a DSL execution, comparing it to the actual observed environment state, and assigning a continuous scalar score representing structural proximity.

Active-Learning Loop: Exploration to Exploitation

The interactive nature of ARC-AGI-3 dictates that the agent must first gather sufficient information to deduce the win condition before shifting to reward maximization. This transition requires a rigorous active-learning loop. The following pseudo-code details the required architectural execution flow:

Python

```
def active_learning_loop(env, max_steps, qwen_llm, dsl_engine):
```

```
    interaction_history = []
```

```
    hypothesis_pool = set
```

```
    for step in range(max_steps):
```

```
        current_state = env.observe()
```

```
        # Calculate epistemic uncertainty using Multi-Head Critic variance
```

```
        uncertainty = calculate_epistemic_uncertainty(hypothesis_pool, current_state)
```

```
        if uncertainty > EXPLORATION_THRESHOLD:
```

```
            # Exploration Phase: Execute actions that maximize information gain (Surprisal)
```

```
            action = maximize_surprisal(qwen_llm, current_state, env.valid_actions)
```

```
            reward, next_state = env.step(action)
```

```
            interaction_history.append((current_state, action, reward, next_state))
```

```
            # Generate new rules using Bottom-Up Synthesis + AB-MCTS
```

```
            # 'wide' mode encourages sampling diverse DSL combinations
```

```
            new_rules = ab_mcts_search(
```

```
                history=interaction_history,
```

```
                llm_evaluator=qwen_llm,
```

```
                mode='wide')
```

```
            hypothesis_pool.update(new_rules)
```

```
        else
```

```
            # Exploitation Phase: Win condition hypothesized with high confidence
```

```
            best_rule = select_minimum_description_length(hypothesis_pool)
```

```
            # Use AB-MCTS to plan the shortest path to the goal state
```

```
            # 'deep' mode refines the selected rule into sequential actions
```

```
            action = ab_mcts_search(
```

```
                history=interaction_history,
```

```
                llm_evaluator=qwen_llm,
```

```
                rule=best_rule,
```

```
                mode='deep')
```

```
            reward, next_state = env.step(action)
```

```
            interaction_history.append((current_state, action, reward, next_state))
```

```
        if assess_progress(current_state, next_state, best_rule):
```

```
            # Update AB-MCTS-A Beta priors (increase alpha)
```

```

        update_beta_parameters(best_rule, success=True)
    else
        # Hypothesis invalidated; penalize Beta priors (increase beta) and revert
        hypothesis_pool_remove(best_rule)
        update_beta_parameters(best_rule, success=False)

    return finalize_trajectory(interaction_history)

```

Pillar 3: Autonomous Exploration & Reinforcement Learning

Deploying standard reinforcement learning algorithms in hidden-state grid worlds frequently results in premature convergence or entropy collapse. The policy rapidly fixates on a local optimum—often repeating a safe, low-reward action—and fails to discover the overarching game mechanics¹⁶. To counteract this, Reinforcement Learning with Verifiable Rewards (RLVR) must be heavily augmented with intrinsic motivation metrics that mathematically compel the model to investigate the unknown¹⁶.

Group Relative Policy Optimization (GRPO) & Calibration Collapse

GRPO has emerged as the definitive critic-free variant utilized for post-training reasoning models. It optimizes by normalizing rewards across a group of G sampled responses to compute relative advantages A_i , circumventing the need for a separate, memory-intensive value network¹⁶. The objective function maximizes the advantage relative to the reference policy:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|O_i|} \sum_{t=1}^{|O_i|} \min(\tilde{r}_{i,t} A_i, \text{clip}(\tilde{r}_{i,t}, 1 - \varepsilon, 1 + \varepsilon) A_i) \right]$$

However, standard GRPO applied to the highly ambiguous tasks in ARC-AGI-3 induces a failure mode known as "calibration collapse." During extended rollouts, the model's confidence progressively decouples from objective correctness. The policy becomes highly confident in incorrect grid predictions, eliminating the stochasticity required to discover novel state transitions¹⁶.

Curiosity-Driven Exploration (CDE)

To prevent calibration collapse and incentivize interaction with novel environmental mechanics, the GRPO objective must be modified via the Curiosity-Driven Exploration (CDE) framework. CDE injects two primary intrinsic bonuses into the reward signal:

Actor Curiosity (Surprisal Bonus) The actor's perplexity serves as a highly accurate signal for underexplored regions of the state space. By dynamically rewarding the LLM for producing

responses with higher perplexity (surprisal) that ultimately evaluate as valid transitions, the system penalizes overconfident errors and actively promotes behavioral diversity¹⁸. The intrinsic actor bonus is defined as:

$$B_{\text{actor}}(q, o) = -\frac{1}{T} \sum_{t=1}^T \log \pi_{\theta}(o_t \mid o_{<t}, q)$$

To maintain optimization stability and prevent the model from generating random noise, the combined reward $\tilde{r}$ is dynamically clipped. The total sentence-level reward combines the extrinsic environment reward $r(q, o)$ and the actor bonus, strictly bounded by a clipping ratio κ and a scaling factor α . Capping the bonus at $|r(q, o)|/\kappa$ ensures the intrinsic reward remains a supplement and cannot reverse the sign of a negative extrinsic penalty¹⁷.

Critic Curiosity (Epistemic Uncertainty) To quantify the uncertainty of the environment itself, the architecture employs a multi-head critic framework. Instead of a single value head, K critics share the LLM backbone but are trained on randomly resampled subsets of the trajectory data¹⁸. The variance across the value estimates of these K heads directly quantifies the epistemic uncertainty of a given board state.

When the variance is high, the model is mathematically incentivized to transition into that state to resolve the uncertainty. This multi-head variance functionally mirrors a count-based exploration bonus, but operates effectively in high-dimensional continuous spaces without requiring an intractable state-visitation table¹⁶. Empirical scaling laws indicate that setting the number of heads to $K \geq 4$ captures the vast majority of the epistemic variance necessary to drive optimal exploration¹⁷. As state-action coverage expands, the head disagreement calibrates, enabling a smooth, mathematically sound transition from exploration to targeted exploitation¹⁷.

Pillar 4: Memory & Context Window Management

Executing the defined neurosymbolic pipeline necessitates rigorous, deterministic management of the LLM's context window. An agent exploring an ARC-AGI-3 environment may easily accumulate hundreds of interaction steps per game, generating massive prompt payloads containing grid serializations, error traces, and hypothesized DSL code².

VRAM Allocation Constraints on the RTX 6000 Ada

The strict compute constraint enforces offline operation on a single NVIDIA RTX 6000 Ada possessing 48GB of VRAM. A 27-billion parameter model like Qwen 3.6 27B, when deployed in FP8 (yielding exactly 1 byte per parameter), will consume approximately 27 to 29 GB of VRAM strictly for its static weights²⁰.

This leaves roughly 19 to 21 GB of VRAM allocated for the Key-Value (KV) cache, Multi-Token Prediction (MTP) overhead, and general inference buffers. The KV cache footprint scales linearly according to the standard formula:

$$\text{KV Cache Size} = 2 \times L \times H_{kv} \times D_{head} \times B$$

Where L is the number of transformer layers, H_{kv} is the number of KV heads (leveraging Grouped-Query Attention), D_{head} is the dimension per head, and B is the batch size²³.

To maximize the context length and accommodate the necessary interaction history without triggering Out-of-Memory (OOM) exceptions, the KV cache itself must be quantized.

Leveraging vLLM's FP8 KV cache feature allows the Qwen 3.6 27B model to sustain a native context length of approximately 200,000 to 256,000 tokens within the 48GB limit. Hardware benchmarks confirm that running the official FP8 quant of Qwen 3.6 27B with FP8 KV cache on a 48GB card achieves over 200k context lengths while suffering negligible accuracy degradation in needle-in-a-haystack evaluations compared to standard BF16 caching²¹.

Context Eviction and the Harness Architecture

Despite a 200k+ token window, continuous active learning loops simulating hundreds of AB-MCTS branches will inevitably exhaust the KV cache. The solution requires a discrete programmatic harness that intermediates between the local vLLM server and the ARC environment. This design is heavily inspired by Tufa Labs' winning "Duck" harness, the successor to Stochastic Goose and the RGB Agent¹⁹.

The architectural setup utilizes an isolated Python Read-Eval-Print Loop (REPL)¹⁹. Instead of injecting the entire raw sequential history of every failed grid movement directly into the LLM prompt, the harness maintains a persistent, compacted log file on the local disk. The Qwen model interacts with the environment exclusively by emitting Python code that reads this log, executes standard grep commands to filter past hypotheses, and issues API calls to the game state¹⁹.

Continuous Eviction Strategy:

1. **State Summarization:** After N exploratory steps, the harness interrupts the primary generation loop and prompts the LLM to write a highly consolidated markdown summary of the confirmed game mechanics (e.g., "Actions 1-3 rigidly translate the blue cluster; Action 4 recolors orthogonal neighbors").
2. **Context Truncation:** The raw, verbose conversational history of the preceding N steps is aggressively evicted and deleted from the active vLLM context.
3. **State Injection:** The LLM's context is repopulated with a highly compressed base prompt. This prompt contains the core system instructions, the summarized mechanics document, the current grid observation, and the REPL execution endpoint. This architecture guarantees the LLM remembers overarching hypotheses indefinitely without

crashing the 48GB VRAM limit¹⁹.

Tool Parser Considerations: When deploying Qwen 3.6 27B via vLLM, the XML parsing logic must be carefully constrained. The default qwen3_xml tool parser template exhibits known edge-cases with premature <stop> token generation if XML tags like <think> and </think> are improperly nested during multi-step reasoning outputs²⁵. The Python harness must explicitly intercept and sanitize these tags, ensuring historical reasoning blocks are hidden or properly terminated before appending the environment's feedback back into the context history²⁵.

Deliverables Required

1. Ranked Algorithms for the Exploration Loop

Based on empirical performance in multi-turn test-time compute workloads, interactive grid search, and LLM scaling laws, the top three specific algorithms required to handle the exploration loop are:

1. **Adaptive Branching Monte Carlo Tree Search (AB-MCTS-A with Beta Priors):** The definitive choice for controlling the exploration-to-exploitation transition. The Beta formulation perfectly maps to the bounded $[0, 1]$ binary success/failure conditions of ARC-AGI hypothesis testing. It prevents the model from wasting compute on deep refinement of fundamentally flawed hypotheses, providing superior scaling for LLM-driven inference over standard fixed-width MCTS¹⁴.
2. **Curiosity-Driven Group Relative Policy Optimization (CDE-GRPO):** Essential for maintaining policy diversity during hidden-state traversal. By utilizing actor perplexity bounding and multi-head critic variance, the agent intrinsically seeks out environmental states that test the boundaries of its current understanding. This directly mitigates the calibration collapse inherent to GRPO in highly ambiguous tasks¹⁶.
3. **Minimum Description Length (MDL) Bottom-Up Synthesis:** Critical for pruning the DSL search space. By evaluating generated programs strictly on their state-compression capability, the system aggressively filters spurious pattern-matching in favor of generalized, robust causal rules. It prevents the agent from memorizing localized grid artifacts and forces true abstraction⁸.

2. Relevant Repositories and Literature

The following resources contain the critical open-source implementations and mathematical proofs for the proposed architecture:

- **Sakana AI TreeQuest (AB-MCTS Implementation):** The core framework for deploying inference-time scaling and adaptive branching tree search algorithms. It provides the necessary Python scaffolding to treat LLM generations as searchable tree nodes.
 - Repository: <https://github.com/SakanaAI/treequest> [cite: 28, 29]
 - ArXiv Paper: *Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search* (arXiv:2503.04412)³⁰.
- **Tufa Labs Duck Harness:** The state-of-the-art interactive Python REPL harness

designed specifically for the ARC-AGI-3 interactive framework. It features optimized context management and eviction mechanics designed for 27B local models.

- Repository: <https://github.com/Tufalabs/duck-harness> [cite: 32]
- **Curiosity-Driven Exploration (CDE) for RLVR:** Details the exact integration of actor perplexity (surprisal) and multi-head critic variance into the GRPO advantage formulation.
- ArXiv Paper: *CDE: Curiosity-Driven Exploration for Efficient Reinforcement Learning in Large Language Models* (arXiv:2509.09675)¹⁷.

3. Step-by-Step Architectural Flowchart

The following flowchart dictates the synchronous communication loop between the ARC-AGI-3 environment, the Python REPL harness, the DSL synthesis engine, and the Qwen 3.6 27B LLM.

[ARC-AGI-3 Interactive Environment]

▲ |
| Action Exec | State Observation (Grid Tensors)
| ▼

+-----+

| DUCK PYTHON HARNESS |

| 1. Intercepts raw grids & translates to text. |
| 2. Maintains full interaction log on disk. |
| 3. Monitors Token Limits (Eviction/Summarization) |
| 4. Sanitizes XML tags (,). |

+-----+

▲ |
| Python Code | Formatted Prompt (State + Rules)
| ▼

+-----+

| QWEN 3.6 27B (vLLM) |

| [Hardware: RTX 6000 (48GB), FP8 Model & KV Cache] |

| 1. Parses XML tool-calls (qwen3_xml). |
| 2. Evaluates Epistemic Uncertainty (Multi-Head). |
| 3. Emits Hypotheses or Executes Exploitation. |

+-----+

▲ |
| Score / Rule | Hypothesis / DSL AST
| ▼

+-----+

| CDE-GRPO + AB-MCTS SEARCH ENGINE |

| 1. Generates programs using TDLC / Bottom-Up. |
| 2. Applies MDL bounds to compress programs. |
| 3. Branches logic: "Go Wider" (Explore) vs. |

| "Go Deeper" (Refine/Exploit). |

+-----+

Works cited

1. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence - arXiv, <https://arxiv.org/html/2603.24621v1>
2. ARC-AGI-3 Dropped -- and Frontier AI Scored Less Than 1% | by ADITHYA GIRIDHARAN, <https://medium.com/@AdithyaGiridharan/arc-agi-3-dropped-and-frontier-ai-scored-less-than-1-90cd70e65a61>
3. ARC Prize 2026 - ARC-AGI-3 Competition, <https://arcprize.org/competitions/2026/arc-agi-3>
4. 1st place solution + code and official documentation - Kaggle, <https://www.kaggle.com/competitions/abstraction-and-reasoning-challenge/writups/icecuber-1st-place-solution-code-and-official-docu>
5. Neural networks for abstraction and reasoning: Towards broad generalization in machines, <https://arxiv.org/html/2402.03507v1>
6. DreamCoder-ARC: Neurosymbolic ARC Solver - Emergent Mind, <https://www.emergentmind.com/topics/dreamcoder-arc>
7. ARC-AGI-2 Technical Report - arXiv, <https://arxiv.org/html/2603.06590v1>
8. MADIL: An MDL-based Framework for Efficient Program Synthesis in the ARC Benchmark - arXiv, <https://arxiv.org/pdf/2505.01081>
9. MADIL: An MDL-based Framework for Efficient Program Synthesis in the ARC Benchmark, <https://arxiv.org/html/2505.01081v1>
10. Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search - arXiv, <https://arxiv.org/pdf/2503.04412>
11. Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search - OpenReview, <https://openreview.net/pdf/e6d2b8dcb02f4dadb940b53aefa65032db4fbd89.pdf>
12. TreeQuest Framework: Adaptive LLM Teams Outperform Individual Models by 30%, <https://neurohive.io/en/frameworks/treequest-framework-adaptive-llm-teams-outperform-individual-models-by-30/>
13. [Literature Review] Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search - Moonlight | AI Colleague for Research Papers, <https://www.themoonlight.io/en/review/wider-or-deeper-scaling-llm-inference-time-compute-with-adaptive-branching-tree-search>
14. Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search - arXiv, <https://arxiv.org/html/2503.04412v1>
15. Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search - arXiv, <https://arxiv.org/html/2503.04412v5>
16. CDE: CURIOSITY-DRIVEN EXPLORATION FOR EFFICIENT REINFORCEMENT LEARNING IN LARGE LANGUAGE MODELS - OpenReview, <https://openreview.net/pdf?id=5rXN5knHKW>

17. CDE: Curiosity-Driven Exploration for Efficient Reinforcement Learning in Large Language Models - arXiv, <https://arxiv.org/html/2509.09675v1>
18. CDE: Curiosity-Driven Exploration for Efficient Reinforcement Learning in Large Language Models - OpenReview, <https://openreview.net/pdf?id=i7LssLK8zi>
19. ARC Prize 2026 - ARC-AGI-3 | Kaggle, <https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/717133>
20. Best Open-Source LLMs to Self-Host in 2026: VRAM Tier Guide | Spheron Blog, <https://www.spheron.network/blog/best-open-source-llms-self-host-2026-vram-guide/>
21. How much VRAM needed for Qwen 3.6 27B Q8 with 262K context? : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1tvluaj/how_much_vram_needed_for_qwen_36_27b_q8_with_262k/
22. Qwen3.6 27B FP8 runs with 200k tokens of BF16 KV cache at 80 TPS on a single RTX 5000 PRO 48GB : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1t46klu/qwen36_27b_fp8_runs_with_200k_tokens_of_bf16_kv/
23. How to Deploy an LLM On-Premise (2026): GPU Sizing & vLLM - Internal Technologies, <https://internal.ai/how-to-deploy-llm-on-premise>
24. Self-Hosted LLMs: GPU Selection, VRAM Requirements & Hosting Guide - GPU Mart, <https://www.gpu-mart.com/guides/self-hosted-llm>
25. Running Qwen3.5-27B on Mixed GPU Setup: From Pain to Success - GitHub, <https://github.com/allanchan339/vLLM-Qwen3-3.5-3.6-chat-template-fix/blob/main/README.md>
26. tool_parsers - vLLM Documentation, https://docs.vllm.ai/en/stable/api/vllm/tool_parsers/
27. What's new - Modular Documentation, <https://docs.modular.com/max/changelog/>
28. Sakana AI Commercializes AB-MCTS in Sakana Marlin, an Enterprise Agent Generating Up to 100-Page Research Reports With Slides - MarkTechPost, <https://www.marktechpost.com/2026/06/15/sakana-ai-marlin/>
29. treequest/README.md at main · SakanaAI/treequest · GitHub, <https://github.com/SakanaAI/treequest/blob/main/README.md>
30. Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search - arXiv, <https://arxiv.org/html/2503.04412v3>
31. Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search, https://proceedings.neurips.cc/paper_files/paper/2025/hash/32dff2e27b0da1fb5f4209216a948544-Abstract-Conference.html
32. Tufalabs/duck-harness: The Duck: ARC-AGI-3 inference ... - GitHub, <https://github.com/Tufalabs/duck-harness>
33. [2509.09675] CDE: Curiosity-Driven Exploration for Efficient Reinforcement Learning in Large Language Models - arXiv, <https://arxiv.org/abs/2509.09675>